4 DECOMPOSITION AND ABSTRACTION

When computer scientists attempt to solve problems by producing algorithms and coding them into programs, they approach the problems in a particular way. This method has been given the name 'computational thinking'. Before they create a structured solution or algorithm and code it into a program, they must define and analyse the problems. This section will introduce two of the **thought processes** they use – decomposition and abstraction.

GENERAL VOCABULARY

thought process the act of using your mind to consider or think about something

LEARNING OBJECTIVES

- Analyse a problem, investigate requirements (inputs, outputs, processing, initialisation) and design solutions
- Decompose a problem into smaller sub-problems
- Understand how abstraction can be used effectively to model aspects of the real world
- Program abstractions of real-world examples

PROBLEM SOLVING

The tasks of a computer scientist include defining and analysing problems; creating structured solutions – algorithms; and coding the solutions into a form that can be implemented by a computer. These tasks are part of what is known as **computational thinking**.

One of the skills required for computational thinking is algorithm design (which we've covered in detail in this unit). If there is a fault in the algorithm design, then the program will not work, however good a coder you are. Two other skills are **decomposition** and **abstraction**.

SUBJECT VOCABULARY

computational thinking the thought processes involved in formulating problems and their solutions so that the solutions are represented in a form that can be effectively carried out by a computer

decomposition breaking a problem down into smaller, more manageable parts, which are then easier to solve

abstraction the process of removing or hiding unnecessary detail so that only the important points remain

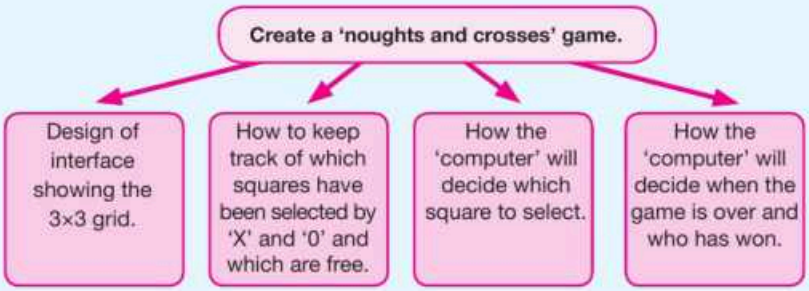
DECOMPOSITION

Decomposition is usually the first step in the problem-solving process. Once a problem has been broken down and the sub-problems have been identified, algorithms can be developed to solve each of them.

Decomposition means that sub-problems can be worked on by different teams at the same time. As smaller algorithms are developed for each sub-problem, it is easier to spot and correct errors. When the algorithm is developed into a program, code can be used again.

WORKED EXAMPLE

A student has been set the task of creating a computer version of the game 'noughts and crosses' (also known as 'tic-tac-toe') where a user plays against the computer.



▶ **Figure 1.12** Sub-problems to be solved to create a noughts and crosses computer program

The diagram shows some of the sub-problems that must be solved in order to solve the complete problem and create a version of the game.

ABSTRACTION

We use abstraction all the time in our daily lives. We **abstract** the essential features of something so that we can understand what people are trying to communicate.

GENERAL VOCABULARY

abstract to remove something from somewhere

Somebody might say, 'I was walking down the street when I saw a cat'. You immediately understand what they mean by 'street' – probably a road with a pavement and houses or shops along the side of it. Similarly, you can picture the cat – a small animal with fur, four legs and a tail. An animal that is basically 'cattish'. You have extracted the basic properties of animals called cats so that you can recognise one when you see one, or imagine one when somebody talks about a cat.



▲ Is this the street and cat you imagined?

What you imagine is very unlikely to match the actual street and cat that the person experienced. But, because of our ability to abstract, the person did not

have to go into unnecessary detail about exactly where they were and what they saw. They wouldn't get very far with the story if they did.

When we create algorithms, we abstract the basic details of the problem and represent them in a way that a computer is able to process.

WORKED EXAMPLE

Yasmin is designing a computer version of a game in which users have to throw a die to find out their number of moves.

In the computer game, the users can't have an actual die, so she will have to design a 'pretend' or virtual die that behaves in exactly the same way as a real-life die.

Yasmin will have to use her powers of abstraction to work out the essential features of a die and then represent them in computer code.

To represent the die, she will have to create a routine that will select a random number from 1 to 6 because that's what a die does.

Yasmin has used abstraction to **model** a real-life event.

GENERAL VOCABULARY

model to make a simple version of something to show how it works

LEVELS OF ABSTRACTION

There are different levels or types of abstraction. The higher the level of abstraction, the less detail is required. We use abstraction all the time in accomplishing everyday tasks.

When programmers write the 'print' command they do not have to bother about all of the details of how this will be accomplished. They are removed from them. They are at a certain level of abstraction.

A driver turning the ignition key to start a car does not have to understand how the engine works or how the spark to ignite the petrol is generated. It just happens and they can simply drive the car. That is abstraction.



▲ A game of noughts and crosses

AN EXAMPLE – NOUGHTS AND CROSSES

Figure 1.12 showed some of the sub-problems that the problem of creating a noughts and crosses game could be divided into. The following could be written at a high level of abstraction.

- The computer goes first. Then the user. This continues until either one wins, or all of the squares have been used.

Immediately a pattern can be recognised – a loop will be needed.

Inputs and outputs

The following inputs from the user will be needed.

- Start the game.
- Entries for the user.
- Select a new game or finish.

The following outputs will be needed.

- A message to inform the user when it is their turn.
- A message to inform the user if they try to select a square that has already been used.

- A message to inform the user if the game is a draw.
- A message to inform the user if they or the computer has won.
- A message to ask the user if they want to play another game or want to finish.

Processing and initialisation

The following processing will be needed.

- Set up the grid with the nine squares.
- Initialise all variables to a start value.
- Decide which square the computer will select.
- Allow the user to select a square.
- Check if the user has selected an already used square.
- Check if the computer or the user has won.
- Check if all squares have been used and the game is a draw.
- Allow the user to select a new game or finish.

The solution is still at a high level of abstraction and more details will need to be added.

For example, the programmer will need to decide how the game will record which player has selected each square; how the computer will decide which square to select; how the game will decide if the computer or the user has won.

The programmer will have to go into more and more detail or move to lower levels of abstraction.

Eventually, the programmer will be able to design an algorithm for the game and code it using a high-level programming language such as Python or Java. Even before they start to implement the game, they will need to plan how they will test the finished program to make sure that it works correctly, what test data they will use and what outcomes it should produce.

CODING AN ALGORITHM

High-level programming languages make it easier for a programmer to write code. Unfortunately, the processor that has to execute the program cannot understand the language it is written in. It therefore needs a translator to translate the code into the only language it does understand – a stream of 1s and 0s.

These high-level languages are therefore at a high level of abstraction – very far removed from the actual language of a computer.

The processing can be split into parts. For example, in the example of the noughts and crosses game there could be separate algorithms for:

- deciding where the computer should make its next selection – it could be called 'computer entry'
- checking if the computer or the player has won – it could be called 'check if won'
- checking if there are any empty squares left – it could be called 'check draw'.

These separate algorithms could be used when they are needed. It is efficient because it means that the same code doesn't have to be rewritten whenever it is needed.

SUBJECT VOCABULARY

subprogram a self-contained module of code that performs a specific task. It can be 'called' by the main program when it is needed

These items of code are called **subprograms**.

In Unit 2, we'll look in detail at how subprograms are used to reduce the complexity of programs and to make them easier to understand.

In the die example above, the designer could write a subprogram called 'die' that generates a random number from 1 to 6. In the main program the designer could just call the 'die' subprogram without having to think about how to implement it each time.

SKILLS REASONING, PROBLEM SOLVING

ACTIVITY 12

CREATING A QUIZ GAME

In a game, each player spins a wheel that is divided into four colours: red, blue, green and yellow. Each player has to answer a question on a particular topic depending on the colour next to a pointer when the wheel stops. Red is for science, blue for history, green for general knowledge and yellow for geography. A player scores two points if they answer correctly on the first attempt and one point for being correct on the second attempt. The first player to reach 30 points is the winner.

Your task is to design a computer version of the game for up to four players. You must analyse the problem and list all of the requirements; **decompose** the problem, list all the sub-problems and write a brief description of each; list all of the input, output and processing requirements.

One of the requirements that will have to be modelled is the spinning of the wheel. Using a written description and pseudocode shows how this could be done.

GENERAL VOCABULARY

decompose to divide something into smaller parts

EXTEND YOUR KNOWLEDGE

Complete the noughts and crosses game by coding it in the language you are studying. See if you can end up with a working game.

CHECKPOINT

Strengthen

SKILLS CRITICAL THINKING

S1 What is meant by 'decomposition'? What are the benefits it provides for programmers?

SKILLS CRITICAL THINKING

S2 What is meant by 'abstraction'?

SKILLS CRITICAL THINKING, PROBLEM SOLVING

S3 A student is creating a model of the cost of a car journey, a real-world problem. Write down the important items she will have to include in her model and how they interact to calculate the cost of the journey.

Challenge

SKILLS CRITICAL THINKING

C1 Can you think of some examples when 'decomposition' and 'abstraction' are used when solving a problem?

SKILLS PROBLEM SOLVING

C2 In your own words, can you explain what is meant by 'computational thinking'?

SKILLS DECISION MAKING

C3 Explain how we use abstraction in our daily lives when we are communicating with others.

DID YOU KNOW?

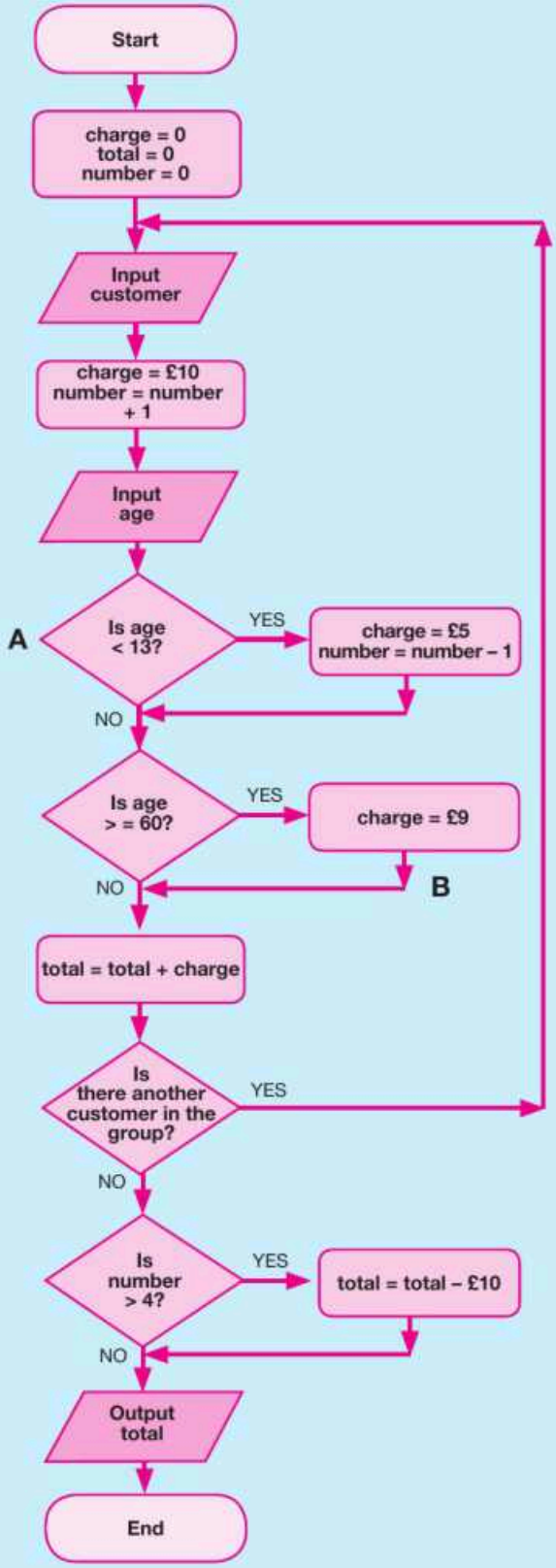
Computer models or 'simulations' of real life are widely used. It is far cheaper and safer to train pilots on flight simulators than on real aircraft. They are also used in weather forecasting, designing and testing new cars and bridges and even teaching people to drive. Computer models are used by all governments around the world to experiment with the short and long-term effects of changing variables such as tax rates on the economy.

How confident do you feel about your answers to these questions? If you're not sure you answered them well, re-read the sections about 'Decomposition' and 'Abstraction'.

SUMMARY

- Computational thinking is an approach to solving problems, such as traffic flow in a city, or how many products a business needs to make and sell to produce a profit. It includes techniques such as decomposition and abstraction.
- Problems are easier to solve if they are decomposed into smaller sub-problems.
- Abstraction is used to remove unnecessary detail to make a problem easier to understand and solve. For example, when modelling traffic flow in a city, unnecessary details could include the colours of the vehicles or the ages of the drivers.
- When designing a solution to a problem the inputs, outputs and processing requirements should be identified at the outset.

UNIT QUESTIONS



▶ **Figure 1.13** Flowchart of an algorithm showing charges in a theme park

UNIT QUESTIONS

The flowchart in Figure 1.13 displays an algorithm used by Holiday Theme Parks Limited.

SKILLS PROBLEM SOLVING A02

1 Explain how the algorithm calculates the total amount that should be paid. **(4)**

SKILLS PROBLEM SOLVING A02

2 Give two variables that are used in the algorithm. **(2)**

SKILLS PROBLEM SOLVING A02

3 In the flowchart, two of the constructs are labelled A and B. State the type of each construct. **(2)**

SKILLS PROBLEM SOLVING A02

4 The Lim family is visiting the park. The family consists of two children, one aged 8 and one aged 10, their two parents and their grandfather, who is aged 65. Use the algorithm to calculate how much the family should have to pay for entry. **(4)**

HINT

Spend some time studying the algorithm to ensure that you fully understand it.

In **1** you are asked to 'explain' how the algorithm works. A longer answer is required, which includes all of the stages of the algorithm. Use the correct terms to explain the constructs.

In **2** and **3** short answers are sufficient.

In **4** calculate the charge for each person using the rules of the algorithm. Then calculate the overall charge and check to see if the family qualifies for a group discount.

SKILLS PROBLEM SOLVING A02

5 A teacher has stored learner surnames as shown below.

Marek	Jackson	Bachchan	Wilson	Abraham	French	Smith

Identify the stages of a bubble sort when applied to this data. **(5)**

SKILLS PROBLEM SOLVING A02

6 The teacher has a sorted list of names from another class as shown below.

Azikiwe	Bloom	Byrne	Davidson	Gateri	Hinton	Jackson	Linton	Smith	Wall

Identify the stages of a binary search to find the name 'Jackson' when applied to this list. **(4)**

HINT

These questions are testing knowledge of the sort and search algorithms.

In question **5**, the answer should be set out to show how the data is progressively sorted using the bubble sort and the result of each pass should be shown.

Question **6** is to check that you know that this is a **recursive method** where the median is repeatedly selected.

SUBJECT VOCABULARY

recursive method a recursive method calls a function over and over until a required goal is met

SKILLS PROBLEM SOLVING A03

7 Create an algorithm to calculate the cost of sending a parcel.

If the weight of the parcel is 2 kg or under then the standard charge is $3. There is then a charge of $2 for each extra kilogram up to 10 kg. After 10 kg the charge per extra kilogram is $3.

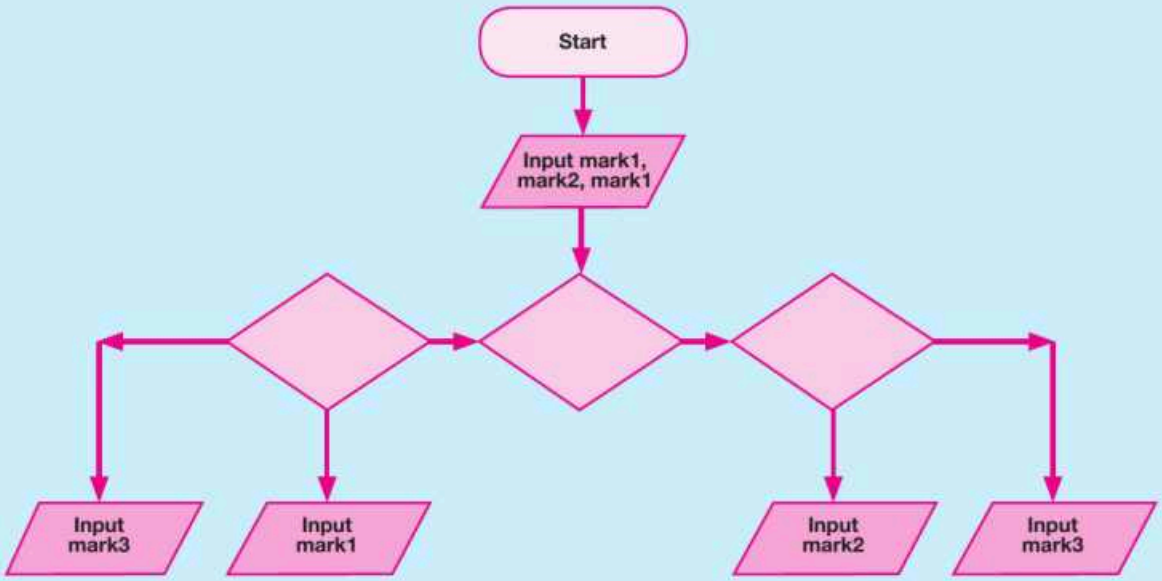
a Display your algorithm as a flowchart. **(5)**

b Construct your algorithm as pseudocode. **(5)**

SKILLS PROBLEM SOLVING A03

8 A learner hands in three homework assignments, which were each given a mark out of 10. All of the marks were different. The following is part of an algorithm to find the highest mark but some of the decision symbols are empty.

Complete the decision symbols and add 'YES' and 'NO' labels where required. **(6)**



SKILLS PROBLEM SOLVING A02

9 A list is made up of the numbers 4, 1, 2, 6, 3, 5.
Identify the steps involved when sorting this list using a bubble sort algorithm. **(2)**